How do you choose which form of this to use?

```
function upcase(text: string): string {
 return text.toUpperCase();
}

const upcase: (text: string) => string = (text) =>
text.toUpperCase();
```

Choosing between a function declaration and a function expression (in this case, an arrow function assigned to a constant) in TypeScript or JavaScript depends on several factors including personal or team coding style, the use case, and certain technical differences. Here's a breakdown to help you decide:

1. Hoisting

- **Function Declaration:** Functions declared this way are hoisted, meaning they are available in their enclosing scope even before they are defined. This allows you to call a function before its declaration in the code.
- **Arrow Function Expression:** Arrow functions are not hoisted because they are assigned to variables. The variables are hoisted but initialized with `undefined` until the code execution reaches the assignment. Thus, you can only use them after the assignment.

2. Syntax and Verbosity

- **Function Declaration:** The syntax is more traditional and can be more readable to those familiar with classical programming languages. It clearly states that this is a function.
- **Arrow Function Expression:** Provides a shorter syntax, which can be less verbose and more modern. It's particularly handy for inline functions or passing functions as arguments.

3. `this` Binding

- **Function Declaration:** The `this` context inside a function declared this way is dynamic. It depends on how the function is called (e.g., via an object method, a simple call, or with `apply` / `call` / `bind`).
- **Arrow Function Expression:** Arrow functions do not have their own `this` context; they inherit `this` from the parent scope at the time they are defined. This is particularly useful in callbacks and methods defined within class-like structures where you want `this` to refer to the outer scope.

4. Use Cases

- **Function Declaration:** Preferred for top-level function definitions and when you need to leverage function hoisting. It's also more suitable when the function needs its own `this` context, such as object methods.
- **Arrow Function Expression:** Ideal for short callbacks, inline functions, and situations where you want to preserve the lexical `this` context, like in event handlers or methods that need to access a class's properties from a callback.

5. Readability and Maintenance

- **Personal or Team Preference:** Some teams might prefer one style over the other for consistency and readability.
- **Project Guidelines:** Follow any established coding standards or guidelines in your project or organization.

Conclusion

- Use a **function declaration** when you need the function to be hoisted, or you rely on its dynamic `this` binding.
- Use an **arrow function expression** for shorter syntax, when you need lexical `this` binding, or for passing functions as arguments.

Ultimately, the choice between these two forms often comes down to the specific needs of your code and personal/team preferences. Both are valid in TypeScript and JavaScript, and understanding the nuances of each can help you make the most appropriate choice for your situation.